

MA3627 Lecture 5 – k -Nearest Neighbours and Linear Regression

The first four weeks of this module were concerned with data in its raw form: how it is acquired, cleaned, transformed, and compressed into a representation suitable for analysis. From this week we turn to the central problem of the module – learning from data. The goal is not merely to describe what a dataset contains, but to extract a function that generalises beyond the observed examples.

This lecture establishes the formal framework that underlies every supervised learning method we will study, and then studies two very different instances of that framework side by side: the k -nearest neighbour classifier, which makes no assumption about the form of the target function, and linear regression, which imposes a strong structural assumption – an affine relationship – and in return admits a closed-form solution. The contrast between a non-parametric and a parametric method, introduced together, is deliberate: it is easier to see what each is buying you, and at what cost, when they sit next to each other. We close with a single treatment of cross-validation that governs hyperparameter choice for both methods, and indeed for every method later in the module.

1 The Supervised Learning Problem

Definition 1.1. A **supervised learning problem** consists of:

- (i) an **input space** $\mathcal{X} \subseteq \mathbb{R}^p$, whose elements are called *feature vectors*;
- (ii) an **output space** $\mathcal{Y}$;
- (iii) an unknown probability distribution P on $\mathcal{X} \times \mathcal{Y}$;
- (iv) a **training set** $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$, where each $(\mathbf{x}_i, y_i)$ is drawn independently from P .

The task is to use $\mathcal{D}$ to construct a function $\hat{f}: \mathcal{X} \rightarrow \mathcal{Y}$ that predicts y accurately for inputs $\mathbf{x}$ not seen during training.

When $\mathcal{Y} = \{1, \dots, C\}$ for some finite C , the problem is called *classification*; when $\mathcal{Y} \subseteq \mathbb{R}$, it is called *regression*. The distinction matters because it governs both which loss functions are natural and what it means for a prediction to be good. Both problems are instances of the same framework, differing only in $\mathcal{Y}$ and the choice of loss function.

We work throughout under the *i.i.d. assumption*: each $(\mathbf{x}_i, y_i)$ is drawn independently from the same distribution P . This fails in time series data and in settings where the input distribution shifts between training and deployment, but it underlies most of the theory in this module and we adopt it as standing background.

1.1 Loss functions and the Bayes predictor

Definition 1.2. A **loss function** is a map $\ell: \mathcal{Y} \times \mathcal{Y} \rightarrow [0, \infty)$ that quantifies the cost of predicting $\hat{y}$ when the true output is y . Common choices include **0–1 loss** for classification, $\ell(y, \hat{y}) = \mathbf{1}[y \neq \hat{y}]$, and **squared loss** for regression, $\ell(y, \hat{y}) = (y - \hat{y})^2$.

The choice of loss function determines which prediction is optimal. Under squared loss, the prediction minimising expected loss at a given $\mathbf{x}$ is the conditional mean $\mathbb{E}[y \mid \mathbf{x}]$. Under 0–1 loss, it is the conditional mode. A function achieving the minimum possible expected loss is a **Bayes predictor**, and the corresponding minimum risk is the **Bayes risk** R^* : the irreducible uncertainty in the problem due to the noise in P , which no learning algorithm can beat in expectation.

Example 1.3. Suppose $\mathcal{Y} = \{0, 1\}$ and $P(y = 1 \mid \mathbf{x}) = \eta(\mathbf{x})$. Under 0–1 loss, the Bayes predictor assigns class 1 if $\eta(\mathbf{x}) > 1/2$ and class 0 otherwise, and $R^* = \mathbb{E}_{\mathbf{x}}[\min(\eta(\mathbf{x}), 1 - \eta(\mathbf{x}))]$. When the same feature vector can genuinely map to either class, $R^* > 0$ and no classifier can achieve zero generalisation error.

1.2 Empirical risk minimisation

The ideal criterion for evaluating $\hat{f}$ is the **risk** $R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim P}[\ell(y, f(\mathbf{x}))]$. Since P is unknown, $R(f)$ cannot be computed directly; what we can compute is the **empirical risk** $\hat{R}(f) = n^{-1} \sum_{i=1}^n \ell(y_i, f(\mathbf{x}_i))$. By the law of large numbers, $\hat{R}(f) \rightarrow R(f)$ as $n \rightarrow \infty$ for any *fixed* function f satisfying the relevant integrability conditions – fixed in the sense of not depending on $\mathcal{D}$ itself. This motivates the following strategy, in which the fitted $\hat{f}$ is instead chosen using $\mathcal{D}$, so the same convergence argument does not apply to it directly.

Definition 1.4. Empirical risk minimisation (ERM) is the strategy of choosing $\hat{f}$ from a specified **hypothesis class** $\mathcal{H}$ to minimise the empirical risk: $\hat{f} = \arg \min_{f \in \mathcal{H}} \hat{R}(f)$.

The hypothesis class $\mathcal{H}$ encodes our assumptions about the problem. Two of the examples we study this week sit at opposite ends of the spectrum: $\mathcal{H}_{\text{lin}} = \{f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b\}$, the class of affine functions underlying linear regression, and $\mathcal{H}_{k\text{-NN}}$, implicitly defined by the k -NN algorithm, where predictions are determined by local majority vote or local averaging and no explicit parametric form is assumed.

Remark 1.5. Minimising $\hat{R}$ over a very large $\mathcal{H}$ can drive the training error to zero while leaving the generalisation error high: a class rich enough to interpolate every training label has learned the noise in $\mathcal{D}$ along with the signal. This is *overfitting*. The gap $R(\hat{f}) - \hat{R}(\hat{f})$ is the *generalisation gap*, and controlling it – by restricting $\mathcal{H}$, by regularisation, or by hyperparameter selection – is a central concern of this and every later lecture.

2 Bias, Variance, and the Decomposition of Generalisation Error

The generalisation error of a learning algorithm depends on two things: how well the best function in $\mathcal{H}$ approximates the true relationship, and how much the learned function fluctuates when $\mathcal{D}$ changes. Fix a test point $\mathbf{x}$ and suppose $y = f^*(\mathbf{x}) + \varepsilon$, where ε is zero-mean noise with variance σ^2 , independent of $\mathbf{x}$ and of the training set $\mathcal{D}$. Let $\hat{f}_{\mathcal{D}}$ denote the function learned from a training set $\mathcal{D}$, and write $\bar{f}(\mathbf{x}) = \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})]$ for the average prediction across all possible training sets of size n .

Theorem 2.1 (Bias–Variance Decomposition). *Under squared loss, the expected generalisation error at $\mathbf{x}$, taken jointly over the training set $\mathcal{D}$ and the independent test-point noise ε , decomposes as*

$$\mathbb{E}_{\mathcal{D}, \varepsilon}[(y - \hat{f}_{\mathcal{D}}(\mathbf{x}))^2 \mid \mathbf{x}] = \underbrace{(f^*(\mathbf{x}) - \bar{f}(\mathbf{x}))^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}_{\mathcal{D}}[(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \bar{f}(\mathbf{x}))^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Irreducible noise}}.$$

Proof. Add and subtract $f^*(\mathbf{x})$ and $\bar{f}(\mathbf{x})$ to write $y - \hat{f}_{\mathcal{D}} = \underbrace{(y - f^*)}_{\varepsilon} + \underbrace{(f^* - \bar{f})}_{\text{const. w.r.t. } \mathcal{D} \text{ and } \varepsilon} + \underbrace{(\bar{f} - \hat{f}_{\mathcal{D}})}_{\text{mean zero over } \mathcal{D}}.$

Square and take the joint expectation $\mathbb{E}_{\mathcal{D}, \varepsilon}[\cdot]$, which integrates out both the randomness in the training set and the independent test-point noise ε ; since ε has mean zero and is independent of $\mathcal{D}$, and $\mathbb{E}_{\mathcal{D}}[\bar{f} - \hat{f}_{\mathcal{D}}] = 0$ by definition of $\bar{f}$, all three cross-product terms vanish under the joint expectation, leaving the three squared terms stated: $\mathbb{E}_{\varepsilon}[\varepsilon^2] = \sigma^2$, the constant $(f^* - \bar{f})^2$, and $\mathbb{E}_{\mathcal{D}}[(\bar{f} - \hat{f}_{\mathcal{D}})^2]$. $\square$

The **bias** measures the systematic error of the algorithm: how far, on average, the learned function is from the truth. The **variance** measures the sensitivity of the learned function to the particular training sample. Bias and variance pull in opposite directions as model complexity changes; the total generalisation error, being their sum, is typically U-shaped in complexity, and identifying the minimum – by cross-validation – is a central task of this lecture and every method we study afterwards.

Example 2.2. Consider estimating a constant $f^*(\mathbf{x}) = \mu$ from n noisy observations $y_i = \mu + \varepsilon_i$, $\varepsilon_i \sim N(0, \sigma^2)$. The sample mean $\bar{y}$ has bias zero and variance σ^2/n . The estimator that always predicts a fixed constant c has squared bias $(\mu - c)^2$ and zero variance. The sample mean has lower expected squared error whenever $\sigma^2/n < (\mu - c)^2$: neither zero bias nor zero variance is the right objective in isolation.

3 k -Nearest Neighbours

The k -nearest neighbour algorithm (k -NN) is among the oldest and most transparent prediction methods. It makes no parametric assumptions about the form of f^* and requires no training phase: the entire training set is retained, and predictions are made at query time by examining local neighbourhoods. For this reason k -NN is called a *lazy* or *instance-based* learner, in contrast to the *eager* learner we study in Section 4, which fits a compact set of parameters during training and discards the data afterwards.

3.1 The algorithm

Definition 3.1. Let $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$ with $\mathcal{Y} = \{1, \dots, C\}$. Fix a positive integer $k \leq n$ and a distance function d . For a query point $\mathbf{x}$, let $\mathcal{N}_k(\mathbf{x})$ denote the indices of the k training points closest to $\mathbf{x}$ under d . The **k -nearest neighbour classifier** predicts $\hat{f}(\mathbf{x}) = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c]$, the plurality vote among the k nearest neighbours. For regression, the prediction is the mean of the k nearest target values, $\hat{f}(\mathbf{x}) = k^{-1} \sum_{i \in \mathcal{N}_k(\mathbf{x})} y_i$.

The most common choice of distance is the Euclidean metric, though the Manhattan and general Minkowski metrics are also used and can matter when features have very different ranges.

Remark 3.2. Because k -NN uses raw feature distances, it is sensitive to the scale of individual features: a feature measured in thousands will dominate the Euclidean distance over one measured in units, regardless of relative importance. Standardising features before applying k -NN is standard practice – one reason the transformation methods of earlier weeks matter directly for model performance.

3.2 Decision boundaries

The **decision boundary** of a classifier is the set of points at which the predicted class changes. For k -NN with $k = 1$ in two dimensions, the *Voronoi diagram* of the training points partitions the plane into regions, one per training point, and each region receives that point's label. The classifier's decision boundary consists of those Voronoi boundaries that separate regions carrying different class labels; a Voronoi edge between two regions belonging to the same class is not itself part of the decision boundary, since the predicted label does not change across it. As k increases, predictions at each point are determined by a larger neighbourhood, and the boundary becomes smoother and less sensitive to individual training points. For $k = n$, the prediction is the majority class everywhere and the boundary disappears. This progression from a jagged, data-hugging boundary to a flat, uninformative one is the geometric manifestation of the bias–variance trade-off.

3.3 The role of k

The integer k is a *hyperparameter*, chosen before training rather than learned from $\mathcal{D}$. When $k = 1$, and assuming no conflicting duplicate feature vectors (two training points with identical $\mathbf{x}_i$ but different y_i), each training observation is its own nearest neighbour, giving zero resubstitution error; the prediction is nonetheless highly sensitive to which particular points make up $\mathcal{D}$, since the identity of the single nearest neighbour at a new query point can change substantially when even a few training points are replaced. As k increases, predictions are averaged over a larger neighbourhood: random label noise has less effect, so variance falls, but the neighbourhood covers more of $\mathcal{X}$ and fine local structure in f^* is smoothed away, so bias rises. Conditional on a fixed neighbourhood and independent homoscedastic noise of variance σ^2 , averaging the k responses in that neighbourhood contributes a noise term of σ^2/k to the variance of the k -NN regressor's prediction at a fixed query point; more generally, increasing k tends to reduce variance while increasing smoothing bias, and the total expected error is minimised at an intermediate k .

Remark 3.3 (Weighted k -NN and computational cost). A natural extension weights each neighbour's vote by $w_i = 1/(d(\mathbf{x}, \mathbf{x}_i) + \epsilon)$ for a small constant $\epsilon > 0$ (needed since w_i would otherwise be undefined when the query point coincides exactly with a training point), so closer neighbours have more influence; this reduces sensitivity to the exact value of k and handles ties more gracefully. On the computational side, a naïve implementation costs $O(np)$ per query; k -d trees and ball trees (both available in `sklearn.neighbors`) reduce this substantially by partitioning $\mathcal{X}$ hierarchically, and are selected automatically based on the data. As with any non-parametric, distance-based method, k -NN suffers from the curse of dimensionality: in high dimensions the neighbourhood needed to contain k points grows rapidly, and locality becomes a weaker notion – a further reason parametric models are often preferred when p is large relative to n .

4 Linear Regression

We now turn to the opposite end of the hypothesis-class spectrum. Linear regression is ERM with the class of affine functions and squared loss. This combination has a closed-form solution and a direct geometric interpretation, and extends naturally to a curved hypothesis class (polynomial regression) and to penalised versions that control variance directly (Section 5).

4.1 Ordinary least squares

For $p > 1$ features, absorb the intercept into the parameter vector by appending a column of ones to the design matrix:

$$X = \begin{pmatrix} 1 & x_{11} & \cdots & x_{1p} \\ \vdots & & & \vdots \\ 1 & x_{n1} & \cdots & x_{np} \end{pmatrix} \in \mathbb{R}^{n \times (p+1)}, \quad \mathbf{w} \in \mathbb{R}^{p+1}, \quad \mathbf{y} \in \mathbb{R}^n.$$

The empirical risk under squared loss is $\hat{R}(\mathbf{w}) = n^{-1} \|\mathbf{y} - X\mathbf{w}\|^2$.

Theorem 4.1 (Normal Equations). *If $X^\top X$ is invertible, the unique minimiser of $\hat{R}(\mathbf{w})$ is $\hat{\mathbf{w}} = (X^\top X)^{-1} X^\top \mathbf{y}$.*

Proof. Expanding, $\hat{R}(\mathbf{w}) = n^{-1}(\mathbf{y}^\top \mathbf{y} - 2\mathbf{w}^\top X^\top \mathbf{y} + \mathbf{w}^\top X^\top X \mathbf{w})$, which is strictly convex when $X^\top X$ is positive definite. Setting $\nabla_{\mathbf{w}} \hat{R} = \mathbf{0}$ gives the normal equations $X^\top X \hat{\mathbf{w}} = X^\top \mathbf{y}$, with the stated unique solution. $\square$

Remark 4.2. $X^\top X$ is invertible if and only if the columns of X are linearly independent. When predictors are nearly collinear, $\hat{\mathbf{w}}$ has very high variance even though the matrix is technically invertible – the phenomenon of *multicollinearity*, and a main motivation for the regularisation of Section 5.

The fitted values $\hat{\mathbf{y}} = X\hat{\mathbf{w}} = H\mathbf{y}$, with $H = X(X^\top X)^{-1}X^\top$ the *hat matrix*, are the orthogonal projection of $\mathbf{y}$ onto the column space of X : H is idempotent and symmetric, and the residual vector $\mathbf{e} = (I - H)\mathbf{y}$ is orthogonal to every column of X , both facts following directly from the definition of H . This picture – prediction as orthogonal projection – connects linear regression directly to the singular value decomposition of Lecture 4: for the *reduced* (economy) SVD $X = U_r \Sigma_r V_r^\top$ of a design matrix X of full column rank $p+1$ (so $U_r \in \mathbb{R}^{n \times (p+1)}$ has orthonormal columns, Σ_r is $(p+1) \times (p+1)$ and invertible, and V_r is $(p+1) \times (p+1)$ and orthogonal), the fitted-value operator is $H = U_r U_r^\top$ and $\hat{\mathbf{w}} = V_r \Sigma_r^{-1} U_r^\top \mathbf{y}$, the Moore–Penrose pseudoinverse applied to $\mathbf{y}$, which is numerically the more stable way to compute $\hat{\mathbf{w}}$ in practice. (Using the full SVD instead would give $UU^\top = I_n \neq H$ in general, since the full U has more columns than X has rank; the reduced SVD, using only the $p+1$ singular vectors associated with non-zero singular values, is what is needed here.)

Theorem 4.3 (Gauss–Markov). *Suppose $\mathbf{y} = X\mathbf{w}^* + \boldsymbol{\varepsilon}$ with $\mathbb{E}[\boldsymbol{\varepsilon} | X] = \mathbf{0}$ and $\text{Cov}(\boldsymbol{\varepsilon} | X) = \sigma^2 I$. Then $\hat{\mathbf{w}}$ is the Best Linear Unbiased Estimator of $\mathbf{w}^*$: among estimators linear in $\mathbf{y}$ and unbiased, $\hat{\mathbf{w}}$ has minimum variance for every linear combination $\mathbf{c}^\top \mathbf{w}^*$.*

This is a limited guarantee: “best among linear unbiased estimators” leaves room for a biased estimator to have lower mean squared error, if its bias is small and its variance reduction is large. Regularised estimators exploit exactly this trade-off.

4.2 Polynomial regression

Polynomial regression extends the hypothesis class by including powers of x as additional features while keeping the model linear in the parameters: $f(x; \mathbf{w}) = w_0 + w_1 x + \dots + w_d x^d = \boldsymbol{\phi}(x)^\top \mathbf{w}$, with feature map $\boldsymbol{\phi}(x) = (1, x, \dots, x^d)^\top$. The normal equations apply without modification; only the design matrix changes, and the same remark applies to any feature map – interactions, logarithms, the polynomial features of earlier weeks all give linear-in-parameters models fitted by OLS.

The degree d is a hyperparameter controlling complexity in exactly the same way k does for k -NN, but in the opposite direction: training error falls (weakly) monotonically as d increases, and for distinct scalar inputs $x_1, \dots, x_n$, a polynomial of degree at most $n-1$ can interpolate all n training observations exactly, so the training error can reach zero at $d = n-1$. (If two observations share the same x value but different y values, no function of x – polynomial or otherwise – can fit both exactly, so this exact interpolation requires the x_i to be distinct.) A high-degree polynomial fitted to a moderate sample can oscillate strongly between training points – low bias, high variance – while a low-degree polynomial is stable but may fit poorly – high bias, low variance. The optimal degree, like the optimal k , is found by cross-validation (Section 6).

5 Regularisation

Regularisation addresses overfitting in linear models directly, by adding a penalty on the size of the slope coefficients to the empirical risk. Large coefficients can be associated with unstable fits, particularly in the presence of multicollinearity or overfitting; penalising coefficient magnitude can therefore reduce variance at the cost of increased bias.

Remark 5.1. For notational simplicity, the formulas below are written as penalising all coordinates of $\mathbf{w}$, including the intercept absorbed into it in Section 4.1. In standard practice and in most software implementations, the intercept is left unpenalised, since it captures the overall level of y rather than model complexity; this is achieved either by centring X and $\mathbf{y}$ before fitting (so the intercept term drops out and only genuine slope coefficients remain to be penalised), or by replacing the penalty matrix I below with $P = \text{diag}(0, 1, \dots, 1)$, giving $\hat{\mathbf{w}}_\lambda = (X^\top X + n\lambda P)^{-1} X^\top \mathbf{y}$ in place of the formula that follows. The same caveat applies to the Lasso formulation in Section 5.2.

5.1 Ridge regression

Ridge regression (ℓ_2 regularisation) minimises $\hat{R}_\lambda(\mathbf{w}) = n^{-1}\|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|^2$ for $\lambda > 0$. The penalised risk is strictly convex for any $\lambda > 0$, and the unique minimiser is

$$\hat{\mathbf{w}}_\lambda = (X^\top X + n\lambda I)^{-1} X^\top \mathbf{y}.$$

The $n\lambda I$ term shifts every eigenvalue of $X^\top X$ upward, making the matrix well-conditioned even when $X^\top X$ is singular, stabilising estimation in the presence of multicollinearity. In terms of the SVD, small singular values – directions poorly determined by the data – are shrunk more aggressively than large ones. Ridge shrinks $\hat{\mathbf{w}}_\lambda$ towards zero as $\lambda \rightarrow \infty$ but does not generally set a coefficient exactly to zero: it performs *shrinkage* but not, in general, *selection*.

5.2 Lasso regression

The **Lasso** minimises $\hat{R}_\lambda(\mathbf{w}) = n^{-1}\|\mathbf{y} - X\mathbf{w}\|^2 + \lambda\|\mathbf{w}\|_1$. Unlike Ridge, the ℓ_1 penalty is not differentiable at zero and there is no closed form; the problem is solved by coordinate descent or the LARS algorithm. Restating both estimators as constrained optimisation problems – Ridge subject to $\|\mathbf{w}\|^2 \leq t$, Lasso subject to $\|\mathbf{w}\|_1 \leq t$ – explains the qualitative difference: the ℓ_2 constraint set is a smooth ball, while the ℓ_1 constraint set is a polytope with corners on the coordinate axes, and the unconstrained minimum is much more likely to first contact a corner than a smooth point. Corners correspond to sparse solutions, so the Lasso performs simultaneous shrinkage and *variable selection*.

Remark 5.2. If many features have small but nonzero effects, Ridge is often effective; if only a few features are relevant, Lasso can provide an interpretable sparse model. In practice, the *elastic net*, combining both penalties, is often used when neither assumption is clearly correct.

6 Model Selection by Cross-Validation

Every method in this lecture has a hyperparameter – k for k -NN, d for polynomial regression, λ for Ridge or Lasso – that controls the position on the bias–variance curve. All of them are selected the same way.

Definition 6.1. In **m -fold cross-validation**, the training set $\mathcal{D}$ is partitioned randomly into m folds of approximately equal size. For each fold j , a model with hyperparameter value θ is trained on the other $m - 1$ folds and evaluated on fold j . The **cross-validation estimate** of generalisation error at θ is

$$\widehat{\text{CV}}(\theta) = \frac{1}{m} \sum_{j=1}^m \hat{R}_j(\hat{f}_{\theta, \mathcal{D} \setminus \mathcal{D}_j}).$$

The value $\hat{\theta}$ minimising $\widehat{\text{CV}}(\theta)$ is selected, and a final model is trained on all of $\mathcal{D}$ using $\hat{\theta}$. The special case $m = n$ is **leave-one-out cross-validation**, which has low bias (each training set differs

from $\mathcal{D}$ by one point) but can have high variance for unstable estimators, since the n training sets overlap heavily. For k -NN specifically, LOOCV has a convenient structure: the held-out point is simply excluded from its own neighbourhood, so it can be computed without refitting from scratch. In practice, 5-fold or 10-fold CV is standard for moderate to large n ; LOOCV is preferred only when n is small enough that every point matters.

6.1 The one-standard-error rule

The curve $\widehat{CV}(\theta)$ is often relatively flat near its minimum, with the minimum itself estimated noisily. The **one-standard-error rule** computes a heuristic estimated standard error $\widehat{SE}(\theta) = m^{-1/2} \text{std}_j(\hat{R}_j)$ across the m folds – heuristic because the fold errors are not independent, since the m training sets overlap, so this is not an exact standard error in the usual i.i.d. sense – and selects the *simplest* model whose cross-validation error lies within one such estimated standard error of the minimum – the largest k , the smallest d , or the largest λ , depending on which hyperparameter is being tuned. This favours a simpler model whose estimated performance is within the sampling uncertainty of the nominal CV minimum, often providing a useful bias toward parsimony, though there is no general guarantee that the resulting model generalises at least as well as the nominal minimiser.

7 Exercises

Exercise 7.1. Let $\mathcal{D}$ consist of six points in $\mathbb{R}^2$ with binary labels:

$$\mathbf{x}_1 = (1, 2), y_1 = 0; \quad \mathbf{x}_2 = (2, 3), y_2 = 0; \quad \mathbf{x}_3 = (3, 1), y_3 = 0;$$

$$\mathbf{x}_4 = (6, 5), y_4 = 1; \quad \mathbf{x}_5 = (7, 4), y_5 = 1; \quad \mathbf{x}_6 = (5, 6), y_6 = 1.$$

For the query point $\mathbf{q} = (4, 4)$, compute the Euclidean distance from $\mathbf{q}$ to each training point and determine the prediction of the k -NN classifier for $k = 1$, $k = 3$, and $k = 5$. Identify any ties and state how they affect the prediction.

Solution. The distances are

$$d(\mathbf{q}, \mathbf{x}_1) = \sqrt{13} \approx 3.61, \quad d(\mathbf{q}, \mathbf{x}_2) = \sqrt{5} \approx 2.24, \quad d(\mathbf{q}, \mathbf{x}_3) = \sqrt{10} \approx 3.16,$$

$$d(\mathbf{q}, \mathbf{x}_4) = \sqrt{5} \approx 2.24, \quad d(\mathbf{q}, \mathbf{x}_5) = 3, \quad d(\mathbf{q}, \mathbf{x}_6) = \sqrt{5} \approx 2.24.$$

Ordering by distance: $\mathbf{x}_2, \mathbf{x}_4, \mathbf{x}_6$ are tied at $\sqrt{5}$; next is $\mathbf{x}_5$ at 3, then $\mathbf{x}_3$ at $\sqrt{10}$, then $\mathbf{x}_1$ at $\sqrt{13}$. For $k = 1$: three training points are tied for nearest neighbour, and these three points include both classes ($\mathbf{x}_2$ is class 0, $\mathbf{x}_4$ and $\mathbf{x}_6$ are class 1), so without a specified tie-breaking rule the prediction is not uniquely determined.

For $k = 3$: the three nearest are $\mathbf{x}_2$ (0), $\mathbf{x}_4$ (1), $\mathbf{x}_6$ (1). The vote is 2–1 for class 1; the classifier predicts class 1.

For $k = 5$: adding $\mathbf{x}_5$ (1) and $\mathbf{x}_3$ (0) gives a 3–2 vote for class 1; the classifier predicts class 1. $\square$

Exercise 7.2. Consider the four observations $(x_1, y_1) = (0, 1)$, $(x_2, y_2) = (1, 3)$, $(x_3, y_3) = (2, 2)$, $(x_4, y_4) = (3, 5)$. Write the design matrix X (with intercept column) and target vector $\mathbf{y}$. Compute $X^\top X$ and $X^\top \mathbf{y}$, solve the normal equations, and state the fitted line. Verify that the residuals sum to zero and are orthogonal to the predictor values.

Solution.

$$X = \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}, \quad \mathbf{y} = \begin{pmatrix} 1 \\ 3 \\ 2 \\ 5 \end{pmatrix}, \quad X^\top X = \begin{pmatrix} 4 & 6 \\ 6 & 14 \end{pmatrix}, \quad X^\top \mathbf{y} = \begin{pmatrix} 11 \\ 22 \end{pmatrix}.$$

From $4\hat{w}_0 + 6\hat{w}_1 = 11$ and $6\hat{w}_0 + 14\hat{w}_1 = 22$: substituting $\hat{w}_0 = (11 - 6\hat{w}_1)/4$ into the second equation gives $16.5 - 9\hat{w}_1 + 14\hat{w}_1 = 22$, so $5\hat{w}_1 = 5.5$, $\hat{w}_1 = 1.1$, and $\hat{w}_0 = (11 - 6.6)/4 = 1.1$.

The fitted line is $\hat{f}(x) = 1.1 + 1.1x$. Fitted values: 1.1, 2.2, 3.3, 4.4. Residuals: $-0.1, 0.8, -1.3, 0.6$, summing to zero. Inner product with predictor values: $0(-0.1) + 1(0.8) + 2(-1.3) + 3(0.6) = 0.8 - 2.6 + 1.8 = 0$, confirming orthogonality to the predictors as well as to the constant. $\square$

Exercise 7.3. Suppose you run 5-fold cross-validation on a k -NN classifier for $k \in \{1, 3, 5, 7, 9\}$ and obtain:

k	$\widehat{CV}(k)$	$\widehat{SE}(k)$
1	0.24	0.04
3	0.18	0.03
5	0.16	0.03
7	0.17	0.03
9	0.19	0.04

Apply the one-standard-error rule. State the minimum error and its standard error, identify all k within one standard error of the minimum, and state which k the rule recommends.

Solution. The minimum is $\widehat{CV}(5) = 0.16$ with $\widehat{SE}(5) = 0.03$, giving a threshold of $0.16 + 0.03 = 0.19$. The values within threshold are $k = 3$ (0.18), $k = 5$ (0.16), $k = 7$ (0.17), and $k = 9$ (0.19); $k = 1$ (0.24) is excluded. Among these, the rule selects the simplest model – the largest k – giving $k = 9$: essentially the same performance as the nominal minimiser, within the estimation noise of the CV procedure, but less likely to overfit the selection criterion itself. $\square$

Further Reading

Lindholm et al. (*SML*, smlbook.org) and James et al. (*ISLP*, statlearning.com) both treat k -nearest neighbours, linear regression, and regularisation as part of a unified introduction to supervised learning, at a level consistent with this lecture; James et al., Chapter 6, is the standard reference for Ridge and the Lasso. Stone (1977) gives foundational consistency results for nonparametric regression, while Cover and Hart (1967) provides the classical asymptotic analysis of nearest-neighbour classification, for students who want the theoretical picture behind the informal bias–variance argument of Section 3.3. Hoerl and Kennard (1970) and Tibshirani (1996) are the original papers introducing Ridge regression and the Lasso respectively.

References

- A. Lindholm, N. Wahlström, F. Lindsten, and T. B. Schön (2022). *Machine Learning: A First Course for Engineers and Scientists*. Cambridge University Press.
- G. James, D. Witten, T. Hastie, R. Tibshirani, and J. Taylor (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer.

- C. J. Stone (1977). Consistent nonparametric regression. *Annals of Statistics*, 5(4), 595–620.
- T. Cover and P. Hart (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- A. E. Hoerl and R. W. Kennard (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- R. Tibshirani (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society, Series B*, 58(1), 267–288.